

RELATÓRIO DE EXECUÇÃO E RESULTADOS DOS TESTES

Projeto: ImperioDelivery

Norma: ISO/IEC/IEEE 29119 | Flutter / Dart | MVVM + Provider

Emitido em: 26/05/2026

1. Identificação do Projeto

Projeto	Imperios App
Tecnologia	Flutter / Dart
Arquitetura	MVVM com Provider
Norma Aplicada	ISO/IEC/IEEE 29119
Tipos de Teste	Unitário e Integração
Frameworks	flutter_test, http/testing, shared_preferences
Data de Execução	26/05/2026
Resultado Geral	104/104 testes aprovados — 100%

2. Objetivo

Registrar a execução dos testes implementados no projeto Flutter Imperios App, documentando os resultados obtidos em cada caso de teste, conforme diretrizes da norma ISO/IEC/IEEE 29119.

3. Ambiente de Execução

Os testes foram executados utilizando o seguinte ambiente:

Componente	Descrição
SDK	Flutter SDK + Dart SDK
Framework de Testes	flutter_test, integration_test
Mocking HTTP	http/testing (MockClient)
Persistência Mock	SharedPreferences (setMockInitialValues)
Arquitetura	MVVM com Provider

4. Estrutura dos Arquivos de Teste

Os testes estão organizados nos seguintes arquivos:

Tipo	Arquivo	Escopo
Unitário	product_model_test.dart	ProductModel — fromJson, toMap, fromMap, toBackendMap
Unitário	product_service_test.dart	ProductService — getAll() com MockClient
Unitário	cart_item_test.dart	CartItem — toMap, fromMap (round-trip)
Unitário	order_model_test.dart	OrderItemModel e OrderModel — fromJson
Integração	auth_service_integration_test.dart	AuthService — login, sessão, endereço, logout, headers
Integração	notification_service_integration_test.dart	NotificationService — FCM, payload, canal Android
Integração	order_service_integration_test.dart	OrderService — finalizarPedido, getOrder, endpoints
Integração	product_service_integration_test.dart	ProductService — getAll() com MockClient (integração)

5. Resultados dos Testes Unitários

Total de casos executados: 39 | Aprovados: 39 | Reprovados: 0

ID	Descrição do Caso de Teste	Resultado Obtido	Arquivo	Status
TC01	fromJson com imagem como Map retorna URL corretamente	ProductModel.image = URL do Cloudinary	product_model	✓ Aprovado
TC02	fromJson com imagem como String retorna URL corretamente	ProductModel.image = URL direta	product_model	✓ Aprovado
TC03	fromJson com imagem null retorna string vazia	ProductModel.image = ""	product_model	✓ Aprovado
TC04	fromJson com valor int converte para double	ProductModel.price = 25.0 (double)	product_model	✓ Aprovado
TC05	fromJson com valor double mantém precisão	ProductModel.price = 19.99	product_model	✓ Aprovado
TC06	fromJson com valor como String numérica converte corretamente	ProductModel.price = 29.90	product_model	✓ Aprovado
TC07	fromJson com valor inválido retorna 0.0	ProductModel.price = 0.0	product_model	✓ Aprovado
TC08	fromJson com id numérico converte para String	ProductModel.id = '42' (String)	product_model	✓ Aprovado
TC09	fromJson sem campo categoria retorna null	ProductModel.categoria = null	product_model	✓ Aprovado
TC10	toMap e fromMap reconstroem o modelo corretamente	Round-trip sem perda de dados	product_model	✓ Aprovado
TC11	toBackendMap retorna mapa com chaves em português	Chaves: produtold, nome, descricao, valor, imagem, categoria	product_model	✓ Aprovado

ID	Descrição do Caso de Teste	Resultado Obtido	Arquivo	Status
TC12	toBackendMap com categoria null retorna campo categoria vazio	categoria = "	product_model	✓ Aprovado
TC13	fromJson sem nome e descricao retorna strings vazias	name = ", description = "	product_model	✓ Aprovado
TC14	retorna lista com 2 produtos quando status é 200	Lista com 2 ProductModel	product_service	✓ Aprovado
TC15	lança Exception quando status é 404	Exception lançada	product_service	✓ Aprovado
TC16	lança Exception quando status é 500	Exception lançada	product_service	✓ Aprovado
TC17	retorna lista vazia sem erro quando data é []	Lista vazia retornada	product_service	✓ Aprovado
TC18	faz requisição para URL contendo /produto	URL path contém /produto	product_service	✓ Aprovado
TC-CI-01	toMap inclui chaves 'product' e 'quantity'	Chaves presentes no mapa	cart_item	✓ Aprovado
TC-CI-02	toMap serializa quantity corretamente	quantity = 5	cart_item	✓ Aprovado
TC-CI-03	toMap serializa product usando toMap() do ProductModel	id = '42', name = 'Pizza'	cart_item	✓ Aprovado
TC-CI-04	quantity padrão é 1 quando não informado	quantity = 1	cart_item	✓ Aprovado
TC-CI-05	fromMap reconstrói CartItem corretamente (round-trip)	Todos os campos preservados	cart_item	✓ Aprovado
TC-CI-06	fromMap com product contendo categoria null reconstrói sem erro	categoria = null, quantity = 2	cart_item	✓ Aprovado
TC-CI-07	fromMap preserva preço com casas decimais	price = 49.99	cart_item	✓ Aprovado
TC-OI-01	fromJson com dados válidos mapeia corretamente	nome, valor, quantidade corretos	order_model	✓ Aprovado
TC-OI-02	fromJson com valor como int converte para double	valor = 40.0 (double)	order_model	✓ Aprovado
TC-OI-03	fromJson com quantidade como double converte para int	quantidade = 3 (int)	order_model	✓ Aprovado
TC-OI-04	fromJson com nome ausente retorna string vazia	nome = "	order_model	✓ Aprovado
TC-OI-05	fromJson com valor null retorna 0.0	valor = 0.0	order_model	✓ Aprovado
TC-OI-06	fromJson com quantidade null retorna 1 como padrão	quantidade = 1	order_model	✓ Aprovado
TC-OM-01	fromJson com dados válidos e itens mapeia corretamente	Todos os campos mapeados	order_model	✓ Aprovado
TC-	fromJson com itens ausentes retorna lista	itens = []	order_model	✓

ID	Descrição do Caso de Teste	Resultado Obtido	Arquivo	Status
OM-02	vazia			Aprovado
TC-OM-03	fromJson com total como int converte para double	total = 50.0 (double)	order_model	✓ Aprovado
TC-OM-04	fromJson com total null retorna 0.0	total = 0.0	order_model	✓ Aprovado
TC-OM-05	fromJson com status ausente usa 'pendente' como padrão	status = 'pendente'	order_model	✓ Aprovado
TC-OM-06	fromJson com id null retorna string vazia	id = "	order_model	✓ Aprovado
TC-OM-07	fromJson com enderecoEntrega e formaPagamento null mantém null	Campos = null	order_model	✓ Aprovado
TC-OM-08	fromJson mapeia corretamente os valores dos itens aninhados	valor=49.0, quantidade=2	order_model	✓ Aprovado

6. Resultados dos Testes de Integração

Total de casos executados: 65 | Aprovados: 65 | Reprovados: 0

ID	Descrição do Caso de Teste	Resultado Obtido	Status
TC-AUTH-01	Login com credenciais válidas retorna null e salva token	isLoggedIn=true, token salvo	✓ Aprovado
TC-AUTH-02	isLoggedIn retorna false quando não há token	isLoggedIn = false	✓ Aprovado
TC-AUTH-03	isLoggedIn retorna false quando token é string vazia	isLoggedIn = false	✓ Aprovado
TC-AUTH-04	Login via HTTP retorna null em sucesso (status 200)	token = null (sem sessão prévia)	✓ Aprovado
TC-AUTH-05	saveEndereco e getEndereco fazem round-trip correto	Endereço recuperado com todos os campos	✓ Aprovado
TC-AUTH-06	getEndereco retorna null quando não há endereço salvo	endereco = null	✓ Aprovado
TC-AUTH-07	getEnderecoFormatado retorna string formatada corretamente	String contém rua, número, cidade, estado	✓ Aprovado
TC-AUTH-08	getEnderecoFormatado retorna null quando não há endereço	formatado = null	✓ Aprovado
TC-AUTH-09	getEnderecos retorna lista vazia quando não há endereços	lista = []	✓ Aprovado
TC-AUTH-10	getEnderecoSelecioneado retorna null quando lista está vazia	selecioneado = null	✓ Aprovado
TC-AUTH-11	setEnderecoSelecioneadoIndex persiste o índice correto	idx = 2	✓ Aprovado
TC-AUTH-12	getEnderecoSelecioneadoIndex retorna 0 como padrão	idx = 0	✓ Aprovado
TC-AUTH-13	Logout limpa token e uid do SharedPreferences	auth_token e auth_uid = null	✓ Aprovado
TC-AUTH-14	Logout remove endereço salvo	user_endereco = null	✓ Aprovado
TC-AUTH-15	authHeaders inclui Authorization quando há token	Authorization: Bearer meu-token-jwt	✓ Aprovado
TC-	authHeaders não inclui Authorization quando não	Chave Authorization	✓ Aprovado

ID	Descrição do Caso de Teste	Resultado Obtido	Status
AUTH-16	há token	ausente	
TC-NS-01	registerToken não deve registrar quando usuário não está logado	isLoggedIn = false, sem registro	✓ Aprovado
TC-NS-02	registerToken pode prosseguir quando usuário está logado	isLoggedIn = true	✓ Aprovado
TC-NS-03	Endpoint de registro de token FCM deve ser POST /user/fcm-token	Contrato validado	✓ Aprovado
TC-NS-04	Body do registro FCM deve conter campo fcmToken	Campo fcmToken presente	✓ Aprovado
TC-NS-05	Header Authorization é incluído ao salvar token FCM	Authorization: Bearer meu-jwt-token	✓ Aprovado
TC-NS-06	Header Authorization ausente quando usuário não está logado	Chave Authorization ausente	✓ Aprovado
TC-NS-07	Payload válido com pedidold é deserializado corretamente	pedidold = 'pedido-xyz-789'	✓ Aprovado
TC-NS-08	Payload sem pedidold retorna null ao acessar campo	pedidold = null	✓ Aprovado
TC-NS-09	Payload malformado não deve lançar exceção não tratada	Exceção capturada internamente	✓ Aprovado
TC-NS-10	Payload null não aciona navegação	payload = null	✓ Aprovado
TC-NS-11	pedidold null não aciona navegação	pedidold = null	✓ Aprovado
TC-NS-12	Dados da mensagem FCM contendo pedidold são extraídos corretamente	pedidold = 'pedido-notificacao-001'	✓ Aprovado
TC-NS-13	Canal Android possui id correto	channelId = 'imperios_orders'	✓ Aprovado
TC-NS-14	Canal Android possui nome correto	channelName = 'Pedidos'	✓ Aprovado
TC-NS-15	Canal Android possui descrição definida	description não vazia	✓ Aprovado
TC-NS-16	Token FCM não é registrado se getToken retorna null	token = null	✓ Aprovado
TC-NS-17	Token FCM é registrado se usuário está autenticado	isLoggedIn=true, token='jwt-valido'	✓ Aprovado
TC-NS-18	onTokenRefresh deve acionar _saveToken com novo token	Contrato de refresh validado	✓ Aprovado
TC-OS-01	Retorna orderId quando backend responde 201 com success=true	orderId='pedido-123', error=null	✓ Aprovado
TC-OS-02	Retorna error quando backend responde com success=false	orderId=null, error='Carrinho vazio'	✓ Aprovado
TC-OS-03	Retorna error de conexão quando há falha de rede	error contém 'conexão'	✓ Aprovado
TC-OS-	finalizarPedido chama endpoint /pedido com	Token presente como pré-	✓ Aprovado

ID	Descrição do Caso de Teste	Resultado Obtido	Status
04	método POST	requisito	
TC-OS-05	finalizarPedido sem endereço e sem forma de pagamento não quebra	body vazio serializado sem erro	✓ Aprovado
TC-OS-06	Body do pedido serializa campos opcionais quando presentes	enderecoEntrega, formaPagamento, observacao presentes	✓ Aprovado
TC-OS-07	getOrder retorna OrderModel completo quando status 200	Todos os campos mapeados corretamente	✓ Aprovado
TC-OS-08	getOrder mapeia itens aninhados corretamente	itens[0].nome='Pizza', valor=49.0	✓ Aprovado
TC-OS-09	getOrder retorna null quando status 404	result = null	✓ Aprovado
TC-OS-10	getOrder retorna null quando há falha de rede	result = null	✓ Aprovado
TC-OS-11	getOrder inclui Authorization no header quando logado	Token 'token-do-usuario' presente	✓ Aprovado
TC-OS-12	getOrder com status diferente de 200 retorna null	result = null	✓ Aprovado
TC-OS-13	Endpoint de criar pedido deve ser POST /pedido	Contrato validado	✓ Aprovado
TC-OS-14	Endpoint de buscar pedido deve ser GET /pedido/:id	path = '/pedido/pedido-abc-123'	✓ Aprovado
TC-OS-15	Simula resposta completa do backend e verifica OrderModel	id='pedido-real-456', status='confirmado', total=98.50	✓ Aprovado
TC-OS-16	Simula resposta com status saiu_para_entrega e verifica mapeamento	status='saiu_para_entrega', total=150.0	✓ Aprovado
TC-PS-01	Retorna lista de ProductModel corretamente com status 200	Lista com 2 produtos	✓ Aprovado
TC-PS-02	Mapeia nome e preço dos produtos corretamente	X-Burguer=25.0, Pizza Margherita=49.90	✓ Aprovado
TC-PS-03	Mapeia imagem Cloudinary (Map com url) para string correta	image = URL Cloudinary	✓ Aprovado
TC-PS-04	Mapeia imagem como String direta corretamente	image = URL direta	✓ Aprovado
TC-PS-05	Imagem null resulta em string vazia no modelo	image = ""	✓ Aprovado
TC-PS-06	Valor int no JSON é convertido para double	price = 3.0 (double)	✓ Aprovado
TC-PS-07	Categoria ausente no JSON resulta em null no modelo	categoria = null	✓ Aprovado
TC-PS-08	Retorna lista vazia quando 'data' é []	Lista vazia	✓ Aprovado
TC-PS-09	Retorna lista com múltiplos produtos mistos	4 produtos	✓ Aprovado

ID	Descrição do Caso de Teste	Resultado Obtido	Status
TC-PS-10	Lança Exception quando status é 401 (não autorizado)	Exception lançada	✓ Aprovado
TC-PS-11	Lança Exception quando status é 404	Exception lançada	✓ Aprovado
TC-PS-12	Lança Exception quando status é 500	Exception lançada	✓ Aprovado
TC-PS-13	Lança Exception quando há erro de rede	Exception lançada	✓ Aprovado
TC-PS-14	Requisição é feita para URL contendo /produto	URL path contém /produto	✓ Aprovado
TC-PS-15	Requisição usa método GET	método = 'GET'	✓ Aprovado

7. Estatísticas Finais

Tipo de Teste	Planejados	Executados	Aprovados	Reprovados
Testes Unitários	39	39	39	0
Testes de Integração	65	65	65	0
TOTAL GERAL	104	104	104	0

8. Análise dos Resultados

8.1 Testes Unitários

Os testes unitários validaram corretamente a camada de modelo e serviço do projeto:

- ProductModel: cobertura completa das variantes de fromJson (imagem como Map/String/null, valor como int/double/String/inválido, id numérico, categoria ausente) e dos métodos toMap, fromMap e toBackendMap.
- CartItem: ciclo round-trip (toMap → fromMap) verificado para todos os campos, incluindo casos com categoria null e preço com casas decimais.
- OrderItemModel e OrderModel: parsing de todos os campos com tratamento de tipos numéricos mistos (int/double), valores null e campos ausentes.
- ProductService: comportamento com MockClient validado para cenários de sucesso (status 200), erros HTTP (401, 404, 500) e falha de rede.

8.2 Testes de Integração

Os testes de integração validaram os contratos entre serviços e infraestrutura:

- AuthService: ciclo completo de sessão (salvar token, verificar login, fazer logout, limpar dados), persistência de endereço único e múltiplos, seleção de índice e geração de headers HTTP com e sem autorização.
- NotificationService: pré-condições de autenticação antes do registro do token FCM, contrato do endpoint POST /user/fcm-token, parsing de payload JSON (pedidoId), tratamento de payloads inválidos e configuração do canal Android.
- OrderService: criação de pedido via POST /pedido com validação dos campos opcionais, busca de pedido via GET /pedido/:id, mapeamento completo do OrderModel a partir de respostas simuladas do backend e tratamento de erros de rede e HTTP.
- ProductService (integração): validação de mapeamento de imagens, conversão de tipos numéricos e tratamento de erros com MockClient injetável.

9. Problemas Encontrados

Nenhuma falha funcional foi encontrada durante a execução dos testes. Todos os 104 casos de teste foram executados com sucesso.

Observação: Os testes de integração do NotificationService não exercitam os plugins nativos (firebase_messaging, flutter_local_notifications) diretamente, pois esses requerem execução em dispositivo físico ou emulador com google-services configurado. Os aspectos testáveis sem plugins nativos — pré-requisitos de autenticação, parsing de payload e contrato de endpoint — foram cobertos integralmente.

10. Conclusão

A execução dos testes demonstrou que o sistema atende aos requisitos funcionais definidos, com 100% dos casos aprovados. A utilização de MockClient para HTTP e de SharedPreferences.setMockInitialValues para persistência garantiu isolamento, velocidade de execução e previsibilidade dos resultados.

A estrutura de testes está alinhada com a norma ISO/IEC/IEEE 29119, cobrindo as dimensões de base conceitual (Documento A), processo de teste (Documento B), técnicas e casos de teste (Documento C), execução e resultados (Documento D) e implementação dos testes de integração (Documento E).